

Firmas Digitales con OpenSSL en Linux

Objetivo

Aprender a generar y usar firmas digitales en Linux con OpenSSL.

El servidor firma mensajes con su clave privada RSA.

El cliente verifica esas firmas usando el certificado del servidor (que contiene la clave pública).

Escenario

El servidor ya tiene su clave privada (server.key) y su certificado autofirmado (server.crt).

El cliente recibe ese certificado y lo usará para verificar la autenticidad de los mensajes firmados.

Pasos de la práctica

EN EL SERVIDOR

Preparar un mensaje

Crea un archivo de texto con el mensaje a enviar:

```
echo "Este es un mensaje firmado digitalmente" > mensaje.txt
```

Firmar el mensaje con la clave privada

```
openssl dgst -sha256 -sign server.key -out mensaje.sig mensaje.txt
```

-sha256 → algoritmo hash usado.

-sign server.key → usa la clave privada del servidor.

-out mensaje.sig → salida con la firma digital.

Enviar el mensaje y su firma al cliente si previamente el cliente ya tiene el certificado

```
scp mensaje.txt mensaje.sig usuario@IP_CLIENTE:/home/usuario/
```

Enviar el mensaje, el certificado y su firma al cliente

```
scp mensaje.txt mensaje.sig server.crt usuario@IP_CLIENTE:/home/usuario/
```

EN EL CLIENTE

Obtener la clave pública del servidor desde el certificado

```
openssl x509 -in server.crt -pubkey -noout > server_pub.pem
```

Verificar la firma

```
openssl dgst -sha256 -verify server_pub.pem -signature mensaje.sig mensaje.txt
```

Si la firma es válida:

Verified OK

Si es inválida (mensaje modificado o clave incorrecta):

Verification Failure

Explicación de los archivos

server.key → clave privada del servidor (solo él la tiene).

server.crt → certificado autofirmado (contiene la clave pública).

server_pub.pem → clave pública extraída del certificado.

mensaje.txt → el mensaje original.

mensaje.sig → firma digital generada con la clave privada.

Flujo Conceptual

El servidor genera un hash SHA-256 del mensaje.

Ese hash se cifra con la clave privada del servidor → esto es la firma digital.

El cliente recibe el mensaje + firma (y el certificado) y con la clave pública del certificado verifica que:

El mensaje no fue alterado.

El autor es realmente el servidor (quien posee la clave privada).

Extensión de la práctica a Realizar

- 1.- Enviar múltiples mensajes y verificar firmas en lote.
- 2.- Modificar el mensaje y comprobar que la verificación falla.
- 3.- Probar con otro certificado distinto para ver que la firma no valida.

1.- Enviar múltiples mensajes y verificar firmas en lote

Supongamos que tienes en el cliente varios archivos y sus firmas correspondientes:

mensaje1.txt mensaje1.sig

mensaje2.txt mensaje2.sig

mensaje3.txt mensaje3.sig

...

Y tienes la clave pública del servidor extraída de su certificado:

server_pub.pem

Usando un script Bash

La manera más sencilla es usar un bucle for que recorra todos los archivos y ejecute la verificación automáticamente:

```
#!/bin/bash

# Carpeta donde están los mensajes y las firmas
CARPETA="./mensajes"

# Clave pública del servidor
CLAVE_PUBLICA="server_pub.pem"

# Recorrer todos los archivos .txt en la carpeta
for archivo in "$CARPETA"/*.txt; do
    # Construir nombre de la firma correspondiente
    firma="${archivo%.txt}.sig"

    # Verificar la firma
    resultado=$(openssl dgst -sha256 -verify "$CLAVE_PUBLICA" -signature "$firma" "$archivo" 2>&1)
```

```

# Mostrar resultado
echo "Archivo: $archivo"
echo "Resultado: $resultado"
echo "-----"
done

```

Explicación del script

El script recorre **todos los archivos .txt** dentro de la carpeta mensajes.

Por cada .txt, busca un .sig con el mismo nombre.

Si algún archivo o firma falta, OpenSSL dará un error en la verificación.

`${archivo%.txt}.sig` → toma el nombre del archivo y reemplaza .txt por .sig.

`openssl dgst -sha256 -verify ...` → verifica la firma de cada archivo.

Se imprime el resultado: "Verified OK" o "Verification Failure" para cada archivo.

Ventajas de la verificación en lote

Permite procesar decenas o cientos de archivos automáticamente.

Fácil de integrar en scripts de automatización o sistemas de mensajería.

Reduce errores humanos al verificar manualmente cada archivo.

Ejemplo de salida del script

```

Archivo: ./mensajes/mensaje1.txt
Resultado: Verified OK
-----
Archivo: ./mensajes/mensaje2.txt
Resultado: Verification Failure
-----
Archivo: ./mensajes/mensaje3.txt
Resultado: Verified OK
-----

```

Pasos para el Script

1.- Guardar el script en un archivo

2.- Da permisos de ejecución al script. Esto hace que el archivo pueda ejecutarse como un programa.

```
chmod +x verificar_lote.sh
```

3.- Crear la carpeta de mensajes

Asegúrate de que la carpeta y los archivos existan:

```
mkdir -p mensajes
```

Copia tus mensajes (mensaje1.txt, mensaje2.txt, ...) y sus firmas (mensaje1.sig, mensaje2.sig, ...) dentro de esa carpeta.

Asegúrate de tener `server_pub.pem` en el mismo directorio que el script o ajusta la variable `CLAVE_PUBLICA` con la ruta correcta.

4.- Ejecutar el script

```
./verificar_lote.sh
```

Verás un resultado como:

```
Archivo: ./mensajes/mensaje1.txt
Resultado: Verified OK
-----
Archivo: ./mensajes/mensaje2.txt
Resultado: Verification Failure
-----
```

Entrega de la práctica: Ampliación – Puntos 2 y 3

Documentar a modo de tutorial, los pasos 1, 2 y 3 de la ampliación de la práctica, mostrando cómo se puede comprobar que la verificación de la firma falla cuando el archivo ha sido alterado o la firma no coincide.

Modificación del archivo firmado

Se modifica el contenido de un mensaje ya firmado (mensaje.txt) para simular un cambio o manipulación.

Esto permite demostrar que la verificación de la firma detecta la alteración.

Verificación de la firma

Se ejecuta el proceso de verificación usando OpenSSL:

```
openssl dgst -sha256 -verify server_pub.pem -signature mensaje.sig mensaje.txt
```

Resultado esperado:

```
Verification Failure
```

Esto confirma que la firma ya no es válida y que el mensaje ha sido modificado desde que fue firmado.

Incluir el paso 1 (creación del mensaje y firma original) como referencia para mostrar la comparación con el resultado correcto (Verified OK) antes de la modificación.